FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Software Engineering
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

	Name	Roll Number	Section	Role in the project
Leader	Abdul Rafay Tariq	24K-3061	BSE-5A	Game engine and rules, attacker AI (A*, minimax with alpha-beta), CSP defender, FastAPI backend, experiment harness
Member 2	Danish Ahmed	24K-3025	BSE-5A	Bayesian network, Naive Bayes classifier and log generator, genetic-algorithm map generator, post-game optimizer, React interface

Project Title	CyberStrike
Project Idea	Idea No. _____from the Project Ideas list ☐ Own idea (needs approval) ☐Yes
Category	Part A — Game ☐Yes Part B — Applied / Data ☐
GitHub Repository URL	https://github.com/Abdul-Rafay-web/CyberStrike
Date of Submission	22-Oct-2026

1. Project Title and Summary

State the title of your project, then summarise it in four to six sentences: what the software does, who would use it or play it, and what AI makes possible in it that a plain program could not do. Write it so that a reader who knows nothing about your topic understands the idea by the end of the paragraph. Avoid listing libraries here.

CyberStrike is a turn-based strategy game on a company network map, where servers are nodes and connections are edges. An Attacker tries to reach the “crown jewel” database. A Defender tries to stop it with patches, firewalls and honeypots, without ever seeing the attacker’s true position. It can be played as Human vs AI, AI vs AI, or two humans. AI is essential because the attacker must plan against a reacting opponent, the defender must infer hidden information from noisy alerts, and every game uses a freshly evolved network.

2. Problem Statement and Motivation

Describe the problem your project addresses in 120–180 words. Why is it hard to solve by hand or by brute force — how large is the search space, how uncertain is the information, how many constraints interact? Who faces this problem, and how do they cope today? Finish with one sentence on why solving it is worthwhile. Be specific — “puzzles are hard,” says nothing; “a 9×9 Sudoku has around 10^21 possible fillings of its empty cells, so blind enumeration is hopeless” says a great deal.

Defending a network is a sequential decision problem against an adaptive, unseen opponent. In a 14-host network the defender places 2 patches, 1 firewall and 1 honeypot each turn: C(14,2) × 12 × 11 = 12,012 legal placements. With about four attacker moves per turn, a three-round lookahead means roughly 10^14 states, so brute force is hopeless. The defender also sees only noisy alerts that miss about 20% of intrusions and fire falsely on about 10% of clean nodes, and resource rules add constraints. Students and analysts usually practise against fixed scripts whose opponent never adapts. A playable simulator with adaptive AI on both sides makes these trade-offs visible and teachable.

3. Problem Formulation

Describe your problem in terms of its agent and environment. Every later design decision follows from this section, so be precise.

3.1 PEAS Description

Component	Description for your project
Performance measure	Attacker: reach the crown jewel quickly and avoid traps (win +100, loss −100, −1 per turn). Defender: prevent the breach until turn 20 or isolate the crown jewel. Zero-sum.
Environment	Simulated 14-node network with entry nodes, a crown jewel and per-node vulnerability scores. Partially observable, stochastic, discrete, two adversarial agents.
Actuators	Attacker: move to an adjacent node or wait. Defender: place 2 patches, 1 firewall and 1 honeypot per turn.
Sensors	Attacker: topology, vulnerability scores, visible defences. Defender: topology, own placements and noisy per-node alerts, but never the attacker’s position unless it enters a honeypot.

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	Fully observable ☐ Partially observable ☑Yes
Determinism	Deterministic ☐ Stochastic ☑Yes
Change over time	Static ☑Yes Dynamic ☐
State and actions	Discrete ☑Yes Continuous ☐
Agents	Single-agent ☐ Multi-agent / adversarial ☑Yes
Agent type	Simple reflex ☐ Model-based ☐ Goal-based ☑Yes Utility-based ☑Yes Learning ☑Yes

3.3 Problem Representation

For a search, game or CSP project, fill every row. For a data-driven project, use the state row for the input features and the goal row for the target you will predict.

Element	Your representation
State representation	Graph with per-node vulnerability, defence status and compromised flag, plus attacker position, turn, remaining resources and side to move. The defender also keeps a belief vector over nodes.
Actions / moves	Attacker: move to an adjacent node or wait. Defender: a valid assignment of patch, firewall and honeypot slots to nodes, or no-op.
Initial state	GA-generated network, attacker on an entry node, no defences, turn 0, uniform belief over entry nodes.
Goal or terminal test	Attacker reaches the crown jewel; or no open path to it remains (defender wins); or turn 20 is reached (defender wins).
Cost, utility, fitness or evaluation	A* edge cost = 1 + 4(1 − vulnerability) + 2 if firewalled. Minimax evaluation is a weighted sum of A* path cost, alternative routes, defences on route and detection belief. GA fitness is closeness to a 50% attacker win rate.

4. Proposed AI Approach

Name each AI technique you will implement and what it does in your project. The core technique must be compared against an alternative — a second technique, a second configuration, or a simple baseline. State whether each is your own code or a library.

#	Technique	Role in the project	Own code or library
1	Minimax with alpha-beta pruning	Attacker's decision engine, with a visible pruned game tree	Own Code
2	A* with hop-count heuristic Genetic algorithm Hill Climbing	Cheapest infiltration route and evaluation input Evolves balanced network layouts Post-game search for a better defender sequence	Own Code Own Code Own Code
3	CSP (backtracking, forward checking, MAC) Bayesian network with Markov blankets Naive Bayes with Laplace smoothing	Defender's legal resource placement Probability of each node holding the attacker, from alerts Classifies events as attack or normal, evaluated with K-fold and a confusion matrixOwn code	Own Code Own Code Own Code
B	Baseline:	Random and greedy attackers, plain minimax, BFS/UCS, random defender, majority-class classifier	Own COde

In 100–150 words, explain why these techniques suit the problem formulated in Section 3, and describe your heuristic, evaluation function, fitness function or probabilistic model in plain words.

cheap route estimates. The Bayesian network handles hidden information, updated from alerts produced by Naive Bayes. The CSP solver turns that belief into a legal placement of defences under limited resources. The GA keeps every map different but balanced, and hill climbing replays finished games to find better defender moves. The A* heuristic is hop distance to the crown jewel, which never overestimates because every step costs at least 1. The minimax evaluation combines A* path cost, alternative routes, defences on the route and detection risk.

4.1 Planned Class Design

List the classes you intend to write — at least three — and for each give its main attributes and methods and its relationship to the others (for example an abstract Agent with Random Agent and AStarAgent subclasses). A simple indented list is acceptable at proposal stage; the full diagram belongs in the report.

GameState: graph, attacker_pos, turn, resources; apply(), is_terminal(), utility() GameEngine: runs turns for two controllers (human or AI) Agent (abstract): choose_action(percept) AttackerAgent: RandomAttacker, GreedyAttacker, AStarAttacker, MinimaxAttacker DefenderAgent: RandomDefender, StaticDefender, CSPDefender HumanAgent: actions come from the React UI AStarSearch, AlphaBetaSearch: search(), nodes_expanded CSPSolver: solve(mode = bt | fc | mac) BeliefTracker: update(alerts), markov_blanket(node) NaiveBayesClassifier: fit(), predict_proba(); KFoldEvaluator NetworkGA: evolve(),

5. Scope

In scope — the features you commit to delivering. List the features that must work for the project to be considered complete.

#	Feature	Why it is essential
1	Game engine and Mode 1 (Human vs AI) in a React graph UI	The Playable Core
2	Attacker AI with A* and alpha-beta minimax, plus game-tree view	Main adversarial-search contribution
3	Naive Bayes alerts, Bayesian belief heatmap and CSP defender	Handles uncertainty and constraints
4	GA map generator and hill-climbing post-game analyser	Varied test maps and a learning feature

5	Mode 2 (AI vs AI) with tournament harness	Produces all experiment results

Out of scope — deliberately excluded. Naming what you will not build is as important as naming what you will; it shows that your scope is realistic for a group of two.

Online multiplayer, real network scanning or packet capture, reinforcement learning, 3D graphics and mobile apps. Human vs Human and Tutorial Mode are stretch goals, started only after Milestone 4.

6. Data or Environment Source

Identify exactly where your data will come from, and confirm that you can actually obtain it. If your project is a game, puzzle or simulation with no external data, tick "Generated / simulated" and describe how problem instances — boards, maps, puzzles, sensor readings — are generated and seeded.

Source name and URL	https://github.com/Abdul-Rafay-web/CyberStrike
Type	Public dataset ☐ Web API ☐ Self-collected ☐ Generated / simulated ☒ Yes
Access requirements	No key needed ☒ Yes Free key required ☐ (Paid sources are not permitted)
Approximate size	30 network layouts (JSON) and 3,000 labelled events (about 70% normal, 30% attack, under 1 MB)
Licence / terms of use	Created by the group, so no third-party terms. Noise and overlap are added so the classifier is not trivially perfect.
Offline fallback	Nothing is downloaded. Everything is generated from fixed seeds and committed to `data/` and `maps/`, and the UI is served locally.

> ***Required:*** your program must still run when the network is unavailable, by reading a locally cached copy of any data it downloads. Describe your caching plan in the last row above.

7. Evaluation Plan

Explain how you will prove that your AI works. Typical metrics are win-rate, average score, nodes expanded, run time, solution cost, accuracy and RMSE. The same instances, the same metric and a fixed seed must be used for every approach you compare.

What will be compared	Alpha-beta vs plain minimax; minimax vs random and greedy attackers; A* vs BFS/UCS; CSP with MAC vs plain backtracking; Bayesian belief vs a last-alert guess; Naive Bayes vs majority class
Metric(s)	Win rate, turns to breach, nodes expanded, run time, belief accuracy, classifier accuracy, precision, recall and F1
Baseline	Random and greedy attackers, plain minimax, BFS/UCS, random defender, majority-class classifier
Test instances / test set	30 fixed GA-generated networks; Naive Bayes on 3,000 events with 5-fold cross-validation
Number of runs and seed	20 games per network per matchup, seed 2002, mean ± standard deviation
Planned charts (at least three)	Win rate by matchup; nodes expanded vs depth (alpha-beta vs minimax); Naive Bayes confusion matrix; belief accuracy by turn; GA fitness by generation

8. Division of Work

State clearly which member is responsible for which part. This table will be compared against your GitHub commit history and against the contribution table in the final report, so make it honest and keep it updated.

Member	Modules / components owned	Approximate share
	Engine, A*, minimax, CSP defender, backend, tournaments	50%
	Naive Bayes, Bayesian network, GA, post-game analyser, React UI, charts	50%

9. Milestones and Timeline

Break the work into weekly or fortnightly milestones from now until the submission deadline in Week 15, one week before the Lab Final Exam. Each milestone should name something demonstrable, not an activity — "A solves all 20 test mazes and prints nodes expanded" rather than "work on search".*

#	Milestone — what will be demonstrably working	Target date	Owner
1	Engine runs a full game; A* solves all 30 test networks and prints nodes expanded; graph renders in React	End of Week 11	Leader/Member 2
2	Alpha-beta attacker beats a random defender with node counts; Naive Bayes passes 5-fold CV with a confusion matrix	End of Week 12	Leader/Member 2
3	Belief heatmap updates each turn; CSP defender places valid resources; Mode 1 fully playable	End of Week 13	Leader/Member 2
4	GA produces the 30 test networks; post-game analyser works; tournaments and all charts generated	End of Week 14	Leader/Member 2
5	Experiments complete, final integration, documentation and repository cleanup	End of Week 15	Both

10. Expected Outcome and Risks

In 80–120 words, describe what a working submission will look like — what a user will see when they run it, and what results you expect to report (for example, "A expands roughly a tenth of the nodes BFS expands"). Then name the two risks most likely to derail the project — for example the search being too slow at the depth you need, or the dataset proving too small to train on — and state what you will do if each occurs.*

The program opens a dark network graph in the browser. The player picks a mode and places defences by clicking nodes while a heatmap shows where the attacker probably is. AI-vs-AI mode plays automatically and shows which branches alpha-beta pruned. We expect alpha-beta to expand about ten times fewer nodes than plain minimax at depth 5, A* to expand far fewer than BFS, the minimax attacker to beat the random and greedy attackers, and Naive Bayes to exceed 90% cross-validated accuracy.

Risk 1 — Minimax too slow. We will use iterative deepening with a time limit, move ordering, and only the top-K CSP-valid defender replies,

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal